

DOCUMENTAÇÃO TÉCNICA

ClassMap

Documentação visual e técnica dos sistemas Elims e GeoCloudAI

Substituto web do Visual Paradigm — diagramas de classes, objetos e visão completa do sistema

Essencislabs

Agosto de 2026

1. Sumário Executivo

O ClassMap é uma ferramenta web, construída sob medida para a Essensilabs, que substitui o Visual Paradigm na tarefa de documentar visualmente os sistemas Elims e GeoCloudAI. Ela nasceu de uma necessidade concreta: apresentar ao gestor uma visão clara e visual da arquitetura dos projetos, no formato que ele mais gosta de consumir — diagramas — sem depender de uma ferramenta de licenciamento antigo, pesada e pouco prática para o time.

Este documento reúne tudo o que foi validado até aqui: a prova de conceito real (extração de 116 classes e 113 relações de um arquivo .vpp do GeoCloudAI), o protótipo funcional com três formas de visualização (classes, objetos e visão do sistema), a arquitetura planejada para produção multi-tenant, a stack tecnológica escolhida e o plano de integração com os agentes de IA (Claude Code e Codex) já usados pelo time no dia a dia.

Principais decisões tomadas

- Nome definitivo do produto: ClassMap.
- Três visualizações no protótipo: Diagrama de Classes, Diagrama de Objetos e Visão do Sistema (nova).
- Notação UML completa: associação, agregação, composição, herança e dependência, com multiplicidade e conectores ortogonais (estilo Visual Paradigm).
- Arquitetura de produção multi-tenant: Organização → Usuários → Projetos → Diagramas, com permissões de dois níveis (visualizador/editor) e administrador por organização.
- Presença em tempo real simplificada: lista de "quem está online" no diagrama, sem cursores ao vivo.
- Personalização visual: cor livre por card, com paleta de referência de ~30 cores predefinidas.
- Stack de produção: React + Vite na Vercel, Supabase (Postgres + Auth + Realtime) — dentro do orçamento de até R\$ 50/mês, com viabilidade de operar no plano gratuito na fase inicial.
- Integração com IA como MVP: os agentes (Claude Code / Codex) geram o arquivo JSON do diagrama a partir do código-fonte; o usuário importa manualmente no ClassMap. Sem automação de CI nesta fase.

2. O Problema e a Motivação

O Visual Paradigm cumpre seu papel, mas traz atrito real para o fluxo de trabalho do time: é uma ferramenta desktop, com licenciamento por posto de trabalho, curva de aprendizado própria e um formato de arquivo (.vpp) proprietário e fechado — na prática, um banco SQLite com um formato de serialização de texto que não é XML nem JSON, e que nenhuma ferramenta de mercado além do próprio Visual Paradigm consegue ler.

Isso cria três problemas práticos para a Essencislabs:

- Compartilhar e atualizar diagramas com o gestor depende de exportar imagens estáticas ou reunir a equipe em uma tela — não existe uma versão "sempre atual" acessível via navegador.
- Não há caminho natural para automatizar a geração dos diagramas a partir do código-fonte real dos projetos Elims e GeoCloudAI.
- O gestor pede recorrentemente uma visão completa do sistema no nível de detalhe de uma planilha técnica (campos, tipos, regras de permissão) — algo que o Visual Paradigm não oferece nativamente.

O ClassMap resolve os três pontos: roda no navegador, guarda os diagramas como um JSON simples e legível (o que abre a porta para geração automática por IA) e ganhou uma visão dedicada de "sistema completo" equivalente à planilha técnica que o gestor já usa como referência.

3. Prova de Conceito: Leitura de Arquivos .vpp Reais

Antes de qualquer decisão de arquitetura, validamos o ponto de maior risco técnico do projeto: seria possível ler os arquivos .vpp existentes do GeoCloudAI e extrair deles as classes e relações reais, sem reconstruir o modelo manualmente?

O formato do arquivo .vpp

Um arquivo .vpp é, por baixo, um banco de dados SQLite. As tabelas relevantes são:

- PROJECT_INFO — metadados do projeto.
- MODEL_ELEMENT — cada classe, atributo, relação e demais elementos do modelo, com o conteúdo real guardado em uma coluna de definição.
- DIAGRAM e DIAGRAM_ELEMENT — quais elementos aparecem em cada diagrama e sua posição visual.

O conteúdo dessa coluna de definição não é XML nem JSON: é um formato de texto proprietário, com uma estrutura do tipo chave-valor aninhada e referências cruzadas entre elementos, por exemplo:

```
4x7f2a1:"DepositType":Class {  Parent=(...);  Child=(4x9b3c2:"name":Attribute {...}, ...); }
```

Para ler isso, desenvolvemos um parser dedicado (tokenizador + analisador recursivo) capaz de interpretar essa gramática, incluindo os casos especiais do formato: blocos de corpo aninhados, objetos sem nome (identificados apenas por tipo) e referências cruzadas entre elementos usando ponteiros internos do arquivo.

Resultado da extração

Rodando o parser contra um arquivo .vpp real do GeoCloudAI ("GeoCloudAI 2 1.vpp"), o resultado foi:

Métrica	Resultado
Classes identificadas	116 de 116 (100%)
Relações identificadas	113 de 113 (100%)
Atributos por classe	Extraídos corretamente, incluindo nomes e tipos
Tipos de relação	Associação, agregação, composição, herança e dependência — todos reconhecidos

Extração 1:1 de um arquivo .vpp de produção — sem perda de classes ou relações.

Esse resultado é a base de tudo o que vem a seguir: como o ClassMap consegue ler o formato real do Visual Paradigm com 100% de fidelidade, a migração dos diagramas já existentes não depende de recriar nada do zero — o mesmo arquivo .vpp pode alimentar o protótipo diretamente.

4. O Protótipo ClassMap

O protótipo já está funcional e navegável, com três modos de visualização acessíveis por abas no topo da tela: Diagrama de Classes, Diagrama de Objetos e Visão do Sistema. Os dados dos 116 elementos extraídos do GeoCloudAI e uma amostra detalhada da planilha técnica GeoCloud.xlsx estão embutidos como exemplo real, para validação com dados de verdade em vez de dados fictícios.

4.1 Diagrama de Classes

É a visão principal: cada classe aparece como um card com nome, estereótipo opcional e a lista de atributos. As relações entre classes são desenhadas como conectores ortogonais (em ângulo reto), no mesmo estilo visual do Visual Paradigm — não curvas suaves, mas segmentos retos que se encontram em cantos de 90°, com um ponto de controle que pode ser arrastado para reorganizar o traçado.



Diagrama de classes completo, com as 116 classes e 113 relações extraídas do arquivo .vpp real do GeoCloudAI.

Notação UML das relações

Cada relação carrega um tipo, escolhido em um seletor visual no painel de edição, com o símbolo geométrico correto aparecendo automaticamente na ponta do conector:

Tipo	Símbolo	Significado
Associação	seta simples	referência entre duas classes, sem posse
Agregação	losango vazado	relação "tem um", mas o filho existe independente do pai
Composição	losango preenchido	relação "é composto por", o filho não existe sem o pai
Herança	triângulo vazado	generalização/especialização entre classes

Tipo	Símbolo	Significado
Dependência	seta tracejada	uma classe depende de outra, sem relação estrutural

Cada relação também aceita multiplicidade opcional nas duas pontas (ex.: "1", "0..*", "n"), exibida junto ao conector — o mesmo padrão de leitura de um diagrama UML tradicional.

ClassMap

Diagrama de ClassesDiagrama de ObjetosVisão do Sistema

Carregar exemplo real (.vpp)Importar JSONExportar JSONLimpar+ Classe

Buscar classe ou objeto...

116CLASSES

113RELAÇÕES

0OBJETO

Account11 attr

AccountInvite8 attr

Address12 attr

AddressType5 attr

Age2 attr

AlterationStyle5 attr

Analysis7 attr

AnalysisAssay11 attr

AnalysisAssayType2 attr

AnalysisMethod3 attr

AnalysisMethodParameter3 attr

AnalysisParameter3 attr

AnalysisStatus3 attr

AnalysisType3 attr

Attachment10 attr

AttachmentMime3 attr

AttachmentType2 attr

Autor2 attr

ChemicalElement5 attr

Class0 attr

Color6 attr

Commodity18 attr

Company9 attr

CompanyType7 attr

Contact8 attr

CoreShed7 attr

Country5 attr

GroupChemical

Age

AnalysisParameter

AttachmentMime

AttachmentType

Attachment

AnalysisStatus

AnalysisAssay

AnalysisMethod

AnalysisMethodParameter

AnalysisParameter

AnalysisType

AttachmentMime

AttachmentType

Attachment

AnalysisStatus

AnalysisAssay

AnalysisMethod

AnalysisMethodParameter

AnalysisParameter

AnalysisType

RELAÇÃO

AnalysisStatus → Analysis

TIPO DE RELAÇÃO

Associação

Agregação

Composição

Herança

Dependência

O símbolo geométrico (losango, triângulo, seta) aparece do lado da classe de origem ou destino conforme a notação UML de cada tipo.

MULTIPLICIDADE (OPCIONAL)

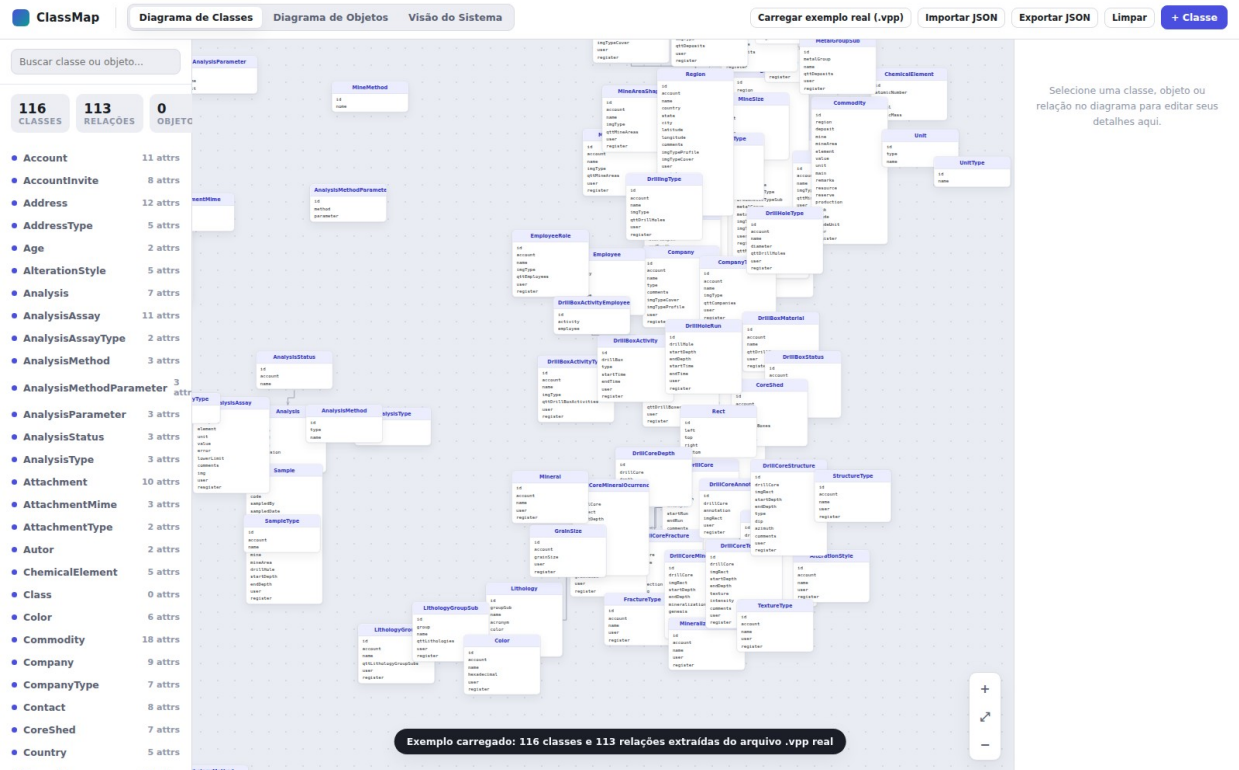
AnalysisStatusAnalysis

ex: 1ex: n

Ajustar curva na tela

Excluir relação

Painel de edição de uma relação: escolha do tipo (com os 5 símbolos UML) e multiplicidade nas duas pontas.



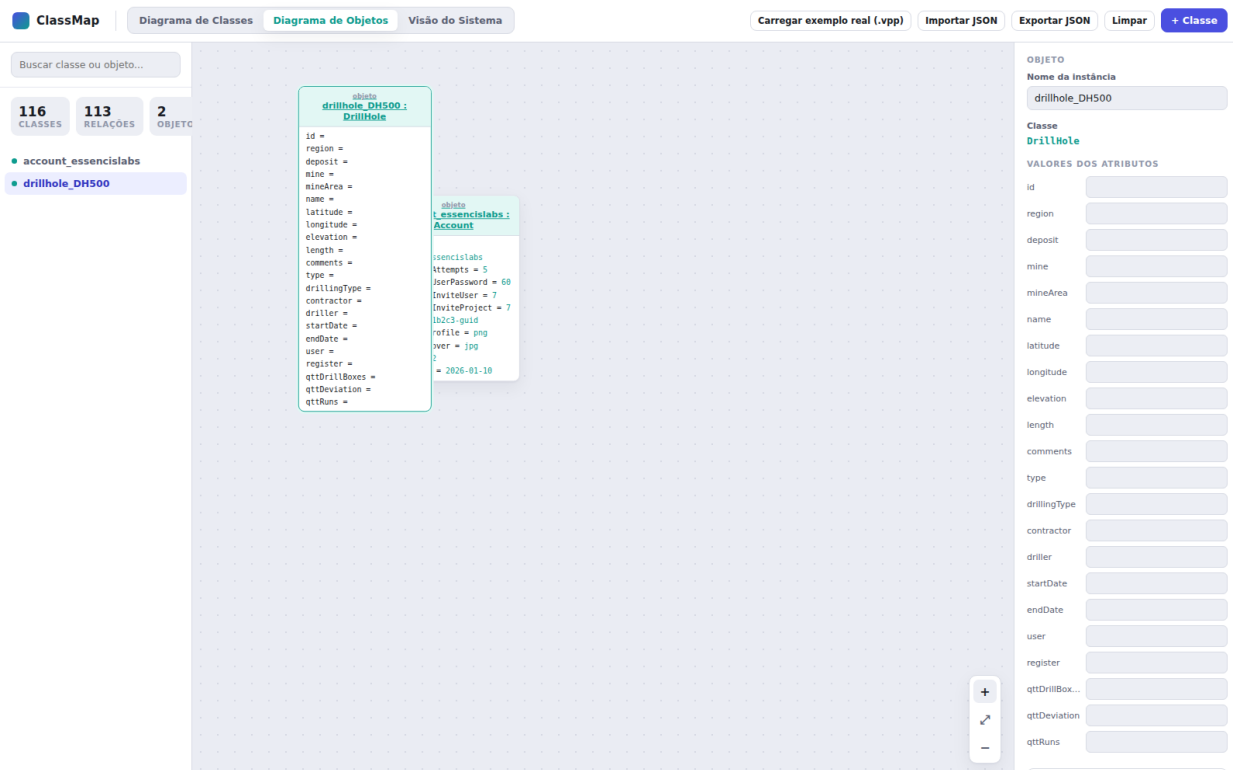
Detalhe aproximado do diagrama, mostrando os conectores ortogonais e os símbolos de relação entre classes do módulo de sondagem.



Painel de edição de uma classe (DrillBox): nome, estereótipo e lista de atributos, totalmente editável.

4.2 Diagrama de Objetos

Complementar ao diagrama de classes, essa visão mostra instâncias concretas — objetos de exemplo com valores reais atribuídos a cada atributo. É útil para ilustrar casos de uso específicos (por exemplo, "como fica um DrillHole cadastrado de verdade") sem misturar dados de exemplo com o modelo estrutural.



Cada objeto pertence a uma classe do diagrama de classes e herda automaticamente a lista de atributos dela — o usuário preenche apenas os valores.

4.3 Visão do Sistema (nova)

Esta é a aba mais recente do protótipo, criada especificamente para resolver o pedido do gestor de enxergar o sistema com o mesmo nível de detalhe de uma planilha técnica completa — a mesma sensação de abrir a planilha GeoCloud.xlsx, mas em formato de navegação visual, dentro do próprio ClassMap.

A tela é organizada por módulo do sistema (ex.: "Core / Autenticação / Administração", "Sondagem / Descrição de Testemunho") e, dentro de cada módulo, por entidade. Ao selecionar uma entidade, três blocos de informação aparecem:

- Campos — uma tabela cruzando todas as camadas do sistema: coluna do banco de dados, tipo, restrições (chave primária, chave estrangeira, autoincremento, obrigatoriedade, unicidade), o tipo no modelo do backend, o tipo no DTO, a regra de validação (obrigatório, tamanho mínimo/máximo) e o tipo no frontend.
- Métodos de API — os métodos expostos pelo controller, com a assinatura correspondente no service e no repository, e o código de permissão vinculado a cada método (quando existir).
- Regras de Permissão — a descrição de cada regra de autorização e a condição de código (C#) que a implementa, exatamente como está documentada na planilha técnica hoje.

ClassMap

Diagrama de ClassesDiagrama de ObjetosVisão do Sistema

Carregar exemplo real (.vpp)Importar JSONExportar JSONLimpar+ Classe

CORE / AUTENTICAÇÃO / ADMINISTRAÇÃO

Account

AccountInvite

User

SONDAGEM / DESCRIÇÃO DE TESTEMUNHO

DrillHole

CORE / AUTENTICAÇÃO / ADMINISTRAÇÃO

Account

12 campos9 métodos de API7 regras de permissão

Campos

Banco → Model → DTO → Front

CAMPO	TIPO (BD)	RESTRIÇÕES	BACK MODEL	DTO	VALIDAÇÃO	FRONTEND
id	int	PK AI NN	int	int	obrigatório	number
name	varchar(40)	NN	string?	string?	obrigatório (4-40)	string
accessMaxAttempts	int	NN	int?	int?	obrigatório (2-10)	number
validityUserPassword	int	NN	int?	int?	obrigatório (10-90)	number
validityInviteUser	int	NN	int?	int?	obrigatório (2-30)	number
validityInviteProject	int	NN	int?	int?	obrigatório (2-30)	number
guid	varchar(36)	NN UQ	string?	string?	opcional	string
imgTypeProfile	varchar(4)	-	string?	string?	opcional (3-4)	string
imgTypeCover	varchar(4)	-	string?	string?	opcional (3-4)	string
userId	int	FK:users	int?	int?	opcional	number
user	-	-	User?	UserDto?	opcional	User
register	dateTime	NN	DateTime?	DateTime?	obrigatório	Date

Métodos de API

Controller · Service · Repository

Add(AccountDto accountDto)
service <ResultService<AccountDto>> Add(AccountDto accountDto, int? userId);
repo <int> Add(Account account, int? userId);

Update(AccountDto accountDto) p1
service <ResultService<AccountDto>> Update(AccountDto accountDto, int? userId);
repo <int> Update(Account account, int? userId);

Delete(int id) p2
service <ResultService<int>> Delete(int accountId, int? userId);
repo <int> Delete(int id, int? userId);

Get([FromQuery] PageParams pageParams) p3
service <Daniel<AccountDto>> Get([FromQuery] PageParams pageParams);

Visão do Sistema — entidade "Account": campos cruzando banco/model/DTO/frontend, métodos de API e regras de permissão.

ClassMap

Diagrama de ClassesDiagrama de ObjetosVisão do Sistema

Carregar exemplo real (.vpp)Importar JSONExportar JSONLimpar+ Classe

CORE / AUTENTICAÇÃO / ADMINISTRAÇÃO

Account

AccountInvite

User

SONDAGEM / DESCRIÇÃO DE TESTEMUNHO

DrillHole

SONDAGEM / DESCRIÇÃO DE TESTEMUNHO

DrillHole

25 campos14 métodos de API8 regras de permissão

Campos

Banco → Model → DTO → Front

CAMPO	TIPO (BD)	RESTRIÇÕES	BACK MODEL	DTO	VALIDAÇÃO	FRONTEND
id	int	PK AI NN	int	int	obrigatório	number
regionId	int	FK:region NN	int	int	obrigatório	number
region	-	-	Region?	RegionDto?	opcional	Region
depositId	int	FK:deposit	int?	int?	opcional	number
deposit	-	-	Deposit?	DepositDto?	opcional	Deposit
mineId	int	FK:mine	int?	int?	opcional	number
mine	-	-	Mine?	MineDto?	opcional	Mine
mineAreaId	int	FK:minearea	int?	int?	opcional	number
mineArea	-	-	MineArea?	MineAreaDto?	opcional	MineArea
name	varchar(40)	NN	string?	string?	obrigatório (4-40)	string
latitude	decimal(10,8)	-	double?	double?	opcional	number
longitude	decimal(11,8)	-	double?	double?	opcional	number
elevation	decimal(10,0)	-	double?	double?	opcional	number
length	decimal(10,0)	-	int?	int?	opcional	number
comments	varchar(40)	-	string?	string?	opcional (4-40)	string
typeId	int	FK:drillholetype	int?	int?	opcional	number
type	-	-	DrillHoleType?	DrillHoleTypeDto?	opcional	DrillHoleType
drillingTypeId	int	FK:drillingtype	int?	int?	opcional	number
drillingType	-	-	DrillingType?	DrillingTypeDto?	opcional	DrillingType
contractorId	int	FK:company	int?	int?	opcional	number
contractor	-	-	Company?	CompanyDto?	opcional	Company
drillerId	int	FK:company	int?	int?	opcional	number
driller	-	-	Company?	CompanyDto?	opcional	Company

Visão do Sistema — entidade "DrillHole" (módulo de Sondagem), com 25 campos e suas chaves estrangeiras destacadas.

ClassMap

Diagrama de Classes

Diagrama de Objetos

Visão do Sistema

Carregar exemplo real (.vpp)

Importar JSON

Exportar JSON

Limpar

+ Classe

CORE / AUTENTICAÇÃO / ADMINISTRAÇÃO

- Account
- AccountInvite
- User

SONDAGEM / DESCRIÇÃO DE TESTEMUNHO

- DrillHole

p5 GETBYID

Permission for users, read self

if (UserIdToken == result.Id) return Ok(result);

p6 GETBYID

Permission for account-owner-entity users, read users of own account

if (OwnerAccountToken == true && result.Entity?.AccountId != null && AccountIdToken == result.Entity.AccountId) return Ok(result);

p7 GETBYID

Permission for users from the system owner entity, read any user

if (EntityIdToken == 1) return Ok(result);

p8 UPDATELAYOUT

Permission for users, edit own layout

if (UserIdToken != layoutDto.Id) return Forbid();

p9 UPLOADIMAGE

Permission for users of own account, read from own account

if (result.Entity?.AccountId != null && result.Entity.AccountId != AccountIdToken) return Forbid();

p10 BLOCK

Permission for users of own account, block on own account

if (target.Entity?.AccountId != null && target.Entity.AccountId != AccountIdToken && EntityIdToken != 1) return Forbid();

p11 RESETATTEMPTS

Permission for users of own account, reset login attempts on own account

if (target.Entity?.AccountId != null && target.Entity.AccountId != AccountIdToken && EntityIdToken != 1) return Forbid();

Regras de permissão da entidade "User", com a condição de código C# exibida junto de cada regra — a mesma informação da planilha técnica, navegável.

Os dados de exemplo desta aba foram extraídos diretamente da planilha GeoCloud.xlsx (entidades Account, AccountInvite, User e DrillHole), provando que o mesmo tipo de conteúdo hoje mantido manualmente em planilha pode ser navegado e mantido dentro do ClassMap.

4.4 Importar e Exportar (formato "diagrama como código")

O ClassMap exporta e importa diagramas em um formato JSON único e simples, pensado desde o início para três usos ao mesmo tempo: backup manual, compartilhamento entre pessoas do time e, principalmente, geração automática por um agente de IA a partir do código-fonte. Este ponto é a base da integração com Claude Code e Codex, detalhada na seção 7.

```
{
  "classes": [ { "name": "User", "attributes": [ {"name": "id", "type": "long"} ] },
  "relationships": [ { "from": "User", "to": "Log", "type": "association" } ],
  "objects": [ { "name": "user1", "class": "User", "values": { "id": "1" } } ] }
```

5. Arquitetura Planejada para Produção

O protótipo validado hoje roda inteiramente no navegador, sem backend. Para virar produto interno usado pelo time e pelo gestor, definimos junto a arquitetura de dados e permissões a seguir.

5.1 Hierarquia multi-tenant

O sistema é organizado em quatro níveis, do mais geral para o mais específico:

Nível	Descrição
Organização	A Essencislabs (ou qualquer outra organização, se o sistema for usado por mais de um cliente no futuro). Cada organização tem um usuário administrador.
Usuários	Pertencem a uma organização. O administrador da organização cria acessos para novos usuários.
Projetos	Pertencem a uma organização (ex.: "Elims", "GeoCloudAI"). O administrador libera, por usuário, quais projetos cada um pode acessar.
Diagramas	Pertencem a um projeto. Podem ser do tipo classes, objetos ou casos de uso.

5.2 Permissões

O controle de acesso é feito em dois níveis por vínculo usuário-projeto, mantendo o modelo simples de operar:

- Visualizador — pode abrir e navegar pelos diagramas do projeto, sem editar.
- Editor — pode criar, editar e excluir diagramas dentro do projeto.

O administrador da organização concede e revoga esse acesso por projeto e por usuário, sem depender de um cadastro de permissões mais granular — o que manteria a complexidade de configuração baixa, alinhado ao uso interno do time.

5.3 Metadados e colaboração

- Cada diagrama registra automaticamente a data da última atualização e o usuário responsável por ela.
- Presença em tempo real: enquanto um diagrama está aberto, o sistema mostra a lista de quais usuários estão visualizando aquele diagrama naquele momento — sem cursores ao vivo estilo Figma, apenas a lista de presença, o que é suficiente para evitar edições simultâneas conflitantes e cobre o cenário de uso do time.
- Personalização visual: cada card de classe ou objeto pode receber uma cor própria, escolhida livremente ou a partir de uma paleta de referência com cerca de 30 cores predefinidas — útil para agrupar visualmente classes por módulo ou destacar áreas do sistema durante uma apresentação.

5.4 Casos de uso (roadmap)

Uma quarta aba de diagramas — "casos de uso", representando o fluxo do sistema do ponto de vista do usuário — está no roadmap, seguindo a mesma lógica visual e de dados dos diagramas de classes e objetos. Ainda não implementada no protótipo atual.

6. Stack Tecnológica e Custo

O critério de escolha foi explícito: subir para produção de forma rápida, prática e com custo baixo ou nulo — teto de R\$ 50/mês. A combinação escolhida evita a necessidade de manter um backend dedicado.

Camada	Escolha	Por quê
Frontend	React + Vite, hospedado na Vercel	Deploy automático a cada push, HTTPS e CDN inclusos, plano gratuito cobre o uso esperado do time.
Backend / dados	Supabase (Postgres gerenciado)	Elimina a necessidade de um servidor próprio; Postgres é robusto e bem documentado.
Autenticação	Supabase Auth	Login pronto (e-mail/senha ou provedores externos), integrado ao controle de acesso por organização.
Tempo real	Supabase Realtime (Presence)	Resolve a funcionalidade de "quem está vendo o diagrama agora" sem gravar isso em uma tabela — é um estado temporário, não persistido.
Segurança de dados	Row Level Security (RLS) do Postgres	A regra "usuário só vê o que sua organização/projeto permite" é garantida no próprio banco, não depende de lógica de aplicação para não vazar dados entre organizações.
Leitura de .vpp	sql.js (SQLite via WebAssembly), no navegador	O arquivo .vpp é lido inteiramente no cliente — nenhum processamento pesado no servidor, nenhum custo adicional de infraestrutura.

Estimativa de custo

Tanto a Vercel quanto o Supabase oferecem planos gratuitos com limites confortáveis para o uso interno de um time — o volume de dados de diagramas e o número de usuários simultâneos previstos ficam dentro desses limites na fase inicial. Caso o uso cresça (mais organizações, mais dados, mais tráfego), o primeiro degrau pago de cada serviço já cobre a operação dentro do teto de R\$ 50/mês definido.

7. Integração com Agentes de IA (Claude Code / Codex)

A motivação central do ClassMap é permitir que os agentes de IA já usados pelo time — Claude Code e Codex — mantenham a documentação visual dos sistemas Elims e GeoCloudAI atualizada, lendo o código-fonte diretamente.

7.1 Escopo definido para o MVP

Para a primeira versão, o escopo foi deliberadamente reduzido: nenhuma automação de CI, nenhuma atualização automática a cada merge na branch principal. O objetivo do MVP é mais simples e direto:

- O agente (Claude Code ou Codex) lê o código-fonte do projeto e gera um arquivo no formato JSON do ClassMap (o mesmo formato de importação/exportação da seção 4.4).
- O usuário sobe esse arquivo manualmente no ClassMap, pelo botão "Importar JSON".
- O ClassMap renderiza corretamente o diagrama de classes e o diagrama de objetos a partir desse arquivo.

Ou seja: automação na geração do conteúdo, mas controle manual sobre quando e o que é publicado — reduzindo o risco de um diagrama incorreto ir parar na tela do gestor sem revisão humana.

7.2 Como o agente decide o tipo de relação

Foi definido um guia de mapeamento código → diagrama, para que diferentes agentes cheguem a resultados consistentes:

Padrão no código	Relação inferida
Classe filha herda/estende a classe pai	Herança
Campo é uma lista de objetos "donos" de outra entidade, sem existir fora do pai	Composição
Campo referencia outra entidade que pode existir de forma independente	Agregação
Chave estrangeira simples / referência de leitura	Associação
Uso pontual de um tipo em um método, sem campo persistente	Dependência
Campo do tipo lista / coleção	Multiplicidade "0..*" ou "n" no lado correspondente

7.3 Diagrama de objetos gerado pelo agente

Para o diagrama de objetos, a orientação ao agente é clara e foi definida como regra explícita: priorizar dados reais de seed/fixtures do projeto quando existirem; na ausência deles, gerar de 1 a 3 exemplos fictícios plausíveis. Em nenhuma hipótese o agente deve incluir dados reais de usuários ou de produção no diagrama de objetos — esse ponto foi fixado como regra de segurança do processo, já documentada no material de apoio entregue ao time (GERAR_DIAGRAMA_CLASSMAP.md).

7.4 Caminho futuro

Uma vez validado o fluxo manual do MVP, o passo natural seguinte — não incluído nesta fase — é acionar esse mesmo processo automaticamente a cada merge na branch principal, publicando a atualização direto no projeto correspondente no ClassMap sem intervenção manual.

8. Roadmap

Etapa	Escopo
Protótipo (concluído)	Diagrama de classes e objetos com notação UML completa, conectores ortogonais, importação/exportação JSON, e a nova Visão do Sistema.
MVP de produção	Migração para React + Vite/Vercel + Supabase, autenticação, hierarquia organização → projeto → diagrama, permissões visualizador/editor.
Colaboração	Metadados de última atualização/autor e lista de presença em tempo real por diagrama.
Personalização	Cor customizável por card (livre ou paleta de ~30 cores).
Casos de uso	Nova aba de diagramas de fluxo do sistema (casos de uso), seguindo o mesmo modelo de dados.
Integração com IA (MVP)	Agente gera o JSON a partir do código-fonte; importação manual no ClassMap.
Integração com IA (avançado)	Geração e publicação automática a cada merge na branch principal, sem intervenção manual.

9. Conclusão

O ClassMap já prova, com dados reais dos projetos Elims e GeoCloudAI, que é possível substituir o Visual Paradigm por uma ferramenta web mais simples, sem perder fidelidade na leitura dos diagramas existentes e ganhando uma visão de sistema completo que hoje só existe em planilha. A arquitetura de produção desenhada mantém o custo dentro do orçamento definido, e o caminho de integração com Claude Code e Codex abre a porta para que a documentação visual do time deixe de ser um trabalho manual recorrente e passe a nascer diretamente do código.

Próximo passo recomendado: validar o MVP de produção (autenticação + hierarquia organização/projeto) com um grupo pequeno do time, antes de expandir para as demais funcionalidades do roadmap.